Practical SAT Solving – WS 2026/27
Prof. Dr. Carsten Sinz

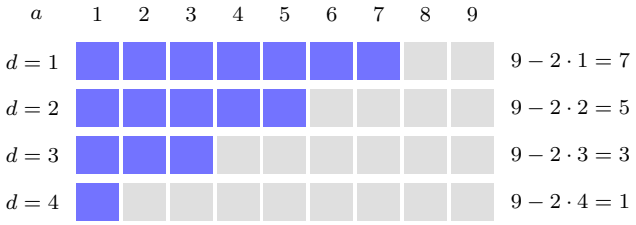
Erlind Sejdiu & Johannes Ott
Hochschule Karlsruhe (HKA)

Übungsblatt 1: Aufgaben 3 & 4

Aufgabe 3: Van-der-Waerden-Zahlen

Die Zahlen $1, \ldots, n$ sollen rot oder blau gefärbt werden, sodass keine k Zahlen mit gleichem Abstand (z. B. $2, 4, 6$) alle dieselbe Farbe haben. Jede Zahl i bekommt eine Variable x_i (rot/blau), also #Variablen $= n$.

So eine Folge $a, a+d, \ldots, a+(k-1)d$ ist durch Startwert a und Abstand d festgelegt. Die Abbildung zeigt alle Folgen für $n = 9$, $k = 3$: Jede Zeile ist ein Abstand d, jedes blaue Kästchen ein möglicher Startwert.



Aus der Abbildung abgelesen: Pro Zeile gibt es $n-(k-1)d$ Startwerte, und es gibt $D = \left\lfloor \frac{n-1}{k-1} \right\rfloor$ Zeilen. Damit:

$$\#\text{Folgen} = \sum_{d=1}^{D} (n-(k-1)d) = nD - (k-1)\sum_{d=1}^{D} d = nD - (k-1)\frac{D(D+1)}{2}$$

Jede Folge liefert 2 Klauseln (nicht alle rot, nicht alle blau):

$$\boxed{\#\text{Klauseln} = 2 \cdot \#\text{Folgen} = 2nD - (k-1)D(D+1), \qquad D = \left\lfloor \frac{n-1}{k-1} \right\rfloor}$$

Probe ($n = 9$, $k = 3$, $D = 4$): $2 \cdot 9 \cdot 4 - 2 \cdot 4 \cdot 5 = 32$ (vgl. Folie 18).

Aufgabe 4: Pythagoräische Tripel

Teil 1: Lösung für die ersten 1000 Zahlen ($n = 1000$)

Die Zahlen $1, \ldots, 1000$ sollen rot oder blau gefärbt werden, sodass bei keinem Tripel mit $a^2 + b^2 = c^2$ alle drei Zahlen dieselbe Farbe haben. Für jedes Tripel werden zwei Klauseln erzeugt: $(x_a \vee x_b \vee x_c)$ und $(\neg x_a \vee \neg x_b \vee \neg x_c)$.

- **Variablen:** 1000
- **Tripel:** 881
- **Klauseln:** $2 \times 881 = 1762$
- **Ergebnis:** Das Problem ist **SAT** (erfüllbar).

Solver-Skript (`Blatt1_Aufgab4.c`):

```c
/*
 * Kompilieren (Beispiel mit CaDiCaL, das IPASIR implementiert):
 *   gcc pythagoras_ipasir.c -I<cadical>/src <cadical>/build/libcadical.a -lstdc++ -lm
 */
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include "ipasir.h"

static void klausel(void *s, int a, int b, int c) {
    ipasir_add(s, a);
    ipasir_add(s, b);
    ipasir_add(s, c);
    ipasir_add(s, 0);
}

int main(int argc, char **argv) {
    int n = argc > 1 ? atoi(argv[1]) : 30;

    void *solver = ipasir_init();               /* Solver anlegen */

    int anzahl = 0;
    for (int a = 1; a <= n; a++) {
        for (int b = a + 1; b <= n; b++) {
            long c2 = (long)a * a + (long)b * b;
            int c = (int)sqrtl((long double)c2);
            if (c > n) break;
            if ((long)c * c != c2) continue;
            klausel(solver,  a,  b,  c);        /* nicht alle blau */
            klausel(solver, -a, -b, -c);        /* nicht alle rot  */
            anzahl++;
        }
    }
    printf("n = %d: %d Tripel, %d Klauseln\n", n, anzahl, 2 * anzahl);

    /* 3. Loesen: 10 = SAT, 20 = UNSAT */
    int ergebnis = ipasir_solve(solver);

    if (ergebnis == 10) {
        printf("SAT Faerbung gefunden\n");
        if (n <= 50) {
            for (int i = 1; i <= n; i++)
                printf("%d: %s\n", i, ipasir_val(solver, i) > 0 ? "rot" : "blau");
        }
    } else if (ergebnis == 20) {
        printf("UNSAT keine Faerbung moeglich\n");
    }

    ipasir_release(solver);                     /* 5. Aufraeumen */
    return 0;
}
```

Programmausgabe für $n = 1000$:

```
n = 1000: 881 Tripel, 1762 Klauseln
SAT Faerbung gefunden
```

Alternative Lösung in Python: Dieses Skript schreibt die Klauseln in eine Datei `problem.cnf`, die man dann mit einem SAT-Solver lösen kann (z. B. `cadical problem.cnf`). Für $n = 1000$ kommen ebenfalls 881 Tripel bzw. 1762 Klauseln heraus.

```python
import math
import sys


def generate_cnf(n, filename):

    """
    Schaut mit jeder (sinnvollen) Kombination aus a und b, ob es ein c gibt,
    das unter n(1000) und ganzzahlig ist. Wenn ja, fuege es zu den Klauseln hinzu
    """

    clauses = []
    for a in range (1, n + 1):
        for b in range (a, n + 1):
            c2 =  a * a + b * b
            c = math.sqrt(c2)
            if (c.is_integer() and c <= n):
                c = int(c)
                clauses.append([a, b, c])

                clauses.append([-a, -b, -c])


    """
    Erstelle eine .cnf Datei mit den Klauseln
    """

    # DIMACS-Datei schreiben
    with open(filename, "w") as f:
        f.write(f"p cnf {n} {len(clauses)}\n")
        for clause in clauses:
            f.write(" ".join(map(str, clause)) + " 0\n")

    print(f"CNF erstellt: {filename}")
    print(f"Variablen:    {n}")
    print(f"Klauseln:     {len(clauses)}")
    print(f"Tripel:       {len(clauses) // 2}")


if __name__ == "__main__":
    n = int(sys.argv[1]) if len(sys.argv) > 1 else 100

    generate_cnf(n, "problem.cnf")
```

Teil 2: Abschätzung von Variablen und Klauseln bis zur Zahl n

Jede Zahl $1 \leq i \leq n$ erhält eine Variable, also $\#\text{Variablen}(n) = n$.

Jedes Tripel ist durch das Paar (a, b) mit $a < b \leq n$ eindeutig bestimmt und liefert 2 Klauseln. Es gibt höchstens $\binom{n}{2}$ solcher Paare, also:

$$\boxed{\#\text{Klauseln}(n) \leq 2\binom{n}{2} = n(n-1) = \mathcal{O}(n^2)}$$